C++: Inheritance

IF2210 – Semester II 2025/2026

Sumber: Diktat Bahasa C++ oleh Hans Dulimarta

Modernisasi oleh Satrio Adi Rukmono

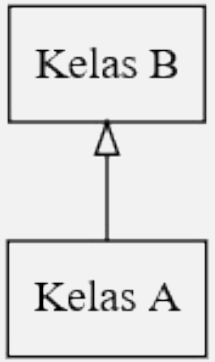
Pewarisan & penurunan kelas

- Pewarisan (inheritance) digunakan untuk menyatakan bahwa sebuah kelas merupakan **bentuk khusus dari kelas lain**.
- Kelas turunan mempertahankan makna dan perilaku dasar kelas induk, serta menyediakan spesialisasi sesuai konteksnya.
- Melalui inheritance, objek dari kelas turunan dapat digunakan di semua situasi yang mengharapkan kelas dasar.

Inheritance di C++ adalah Realisasi Teknis dari Konsep OO

- Pada bagian sebelumnya, inheritance dipahami sebagai relasi:
  - general → specific dalam domain,
  - substitusi objek yang mempertahankan makna.
- Dalam C++, relasi tersebut diwujudkan melalui:
  - deklarasi kelas turunan,
  - virtual function,
  - aturan konstruksi dan destruksi,
  - dynamic binding.
- Bagian ini menjelaskan bagaimana bahasa C++ memastikan relasi tersebut bekerja dengan benar di tingkat program.

Pewarisan & penurunan kelas



- Kelas turunan dibangun di atas kelas dasar:
  - mempertahankan kontrak perilaku kelas dasar,
  - dapat menggunakan implementasi yang sudah ada,
  - menambahkan atau menyesuaikan perilaku sesuai spesialisasi.
- Dalam implementasi C++, bagian dari kelas dasar menjadi **sub-objek** di dalam kelas turunan.

Penurunan kelas dalam C++

```
class ‹kelas-turunan›: ‹mode-pewarisan› ‹kelas-dasar› {
  // ...
};
```

- *Mode-pewarisan*: mempengaruhi tingkat pengaksesan setiap anggota (method & atribut) kelas dasar jika diakses melalui fungsi/method **di luar** kelas dasar maupun di luar kelas turunan.
- Contoh:

```
class Minibus: public Kendaraan {
  // ...
};
```

Penurunan kelas dalam C++

- Perubahan tingkat pengaksesan akibat pewarisan →
- private: = “sangat tertutup” (hanya method kelas tersebut yang dapat mengakses,
- public: = “sangat terbuka” (fungsi / method manapun, di dalam atau di luar kelas dapat mengakses anggota dalam bagian ini),
- protected: “setengah terbuka” / “setengah tertutup” (hanya kelas turunan yang dapat mengakses anggota pada bagian ini).

Tingkat akses di base class	Mode pewarisan		
	private	protected	public
private	private	private	private
protected	private	protected	protected
public	private	protected	public

Contoh pewarisan

- `NotificationSender`: kelas dasar yang merepresentasikan objek yang dapat mengirim notifikasi
  - `send()`: mengirim pesan ke penerima
- `RetryingSender`: pengirim notifikasi yang mencoba ulang jika pengiriman gagal
  - `send()`: mencoba mengirim pesan hingga sejumlah percobaan tertentu
- RetryingSender merepresentasikan *variasi perilaku* dari `NotificationSender`
- Kelas `RetryingSender` dapat diwariskan dari kelas `NotificationSender` dengan:
  - Mengubah perilaku `send()` agar melakukan percobaan berulang
  - Menambahkan atribut untuk menyimpan jumlah maksimum percobaan (`maxRetries`) sebagai konsekuensi perilaku baru

```cpp
class NotificationSender {
  public:
    NotificationSender(std::string id)
        : id_(std::move(id)) {}

    virtual ~NotificationSender() = default;

    virtual void send(const std::string& recipient,
                      const std::string& message) {
        std::cout << "[" << id_ << "] "
                  << "Send to " << recipient
                  << ": " << message << std::endl;
    }

  protected:
    std::string id_;
};
```

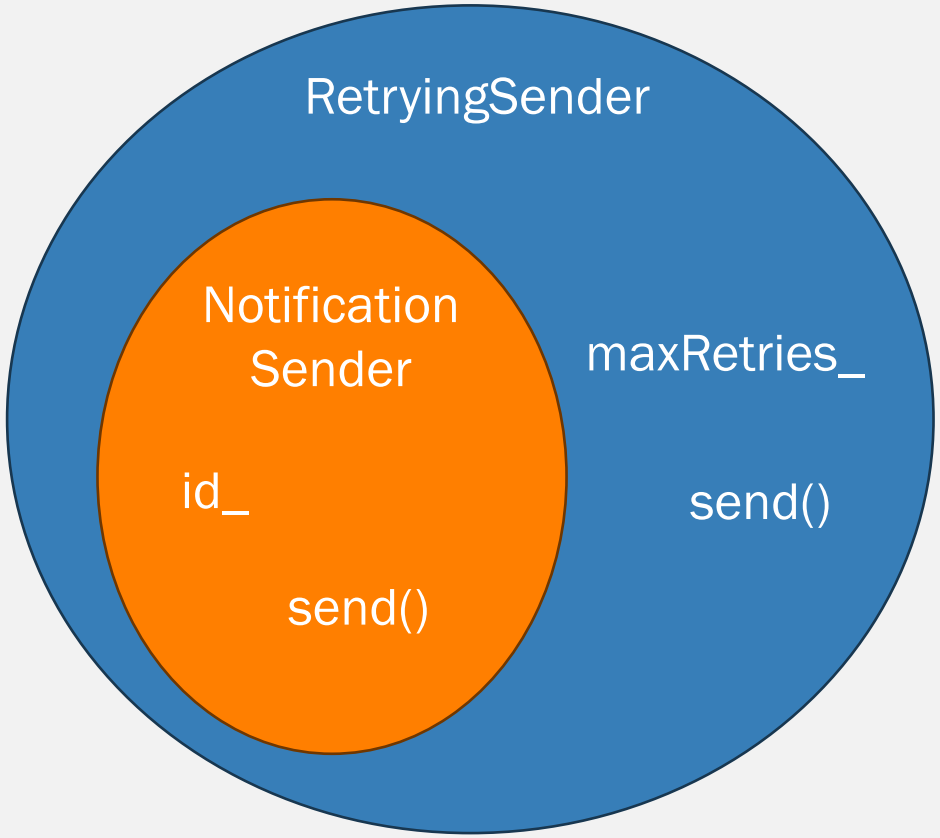
```
class RetryingSender : public NotificationSender {
  public:
    RetryingSender(std::string id, int maxRetries)
        : NotificationSender(std::move(id)),  // inisialisasi base
          maxRetries_(maxRetries) {}

    void send(const std::string& recipient,
              const std::string& message) override {
        for (int attempt = 1; attempt <= maxRetries_; ++attempt) {
            std::cout << "Attempt " << attempt << "..." << std::endl;
            NotificationSender::send(recipient, message);
            // ...
        }
    }

  private:
    int maxRetries_;
};
```

ctor, dtor, cctor, dan `operator=` kelas dasar

- Komponen yang berasal dari kelas dasar dapat dianggap sebagai “sub-objek” dari kelas turunan
- Pada penciptaan objek kelas turunan, konstruktor kelas dasar akan diaktifkan **sebelum** konstruktor kelas turunan.
- Pada pemusnahan objek kelas turunan, destruktor kelas dasar dipanggil **setelah** destruktor kelas turunan.



Lifecycle Objek dalam Hierarki

- Ketika objek turunan dibuat:
  - bagian kelas dasar dibangun terlebih dahulu,
  - kemudian bagian spesifik turunan.
- Ketika objek dihancurkan:
  - bagian turunan dilepas terlebih dahulu,
  - lalu bagian kelas dasar.
- Urutan ini menjaga agar setiap objek selalu berada dalam keadaan valid selama siklus hidupnya.

Penanganan *copy constructor*

- Ada tiga kasus:
  1. Kelas turunan tidak memiliki cctor, kelas dasar memiliki
  2. Kelas turunan memiliki cctor, kelas dasar tidak memiliki
  3. Baik kelas turunan maupun kelas dasar memiliki cctor
- Kasus (1): cctor kelas dasar akan dipanggil, inisialisasi kelas turunan dilakukan secara *bitwise copy*
- Pada kasus (2) dan (3), cctor dari kelas dasar **tidak dipanggil**, inisialisasi kelas dasar menjadi tanggung jawab kelas turunan.

Penanganan *copy constructor*

- Penginisialisasian kelas dasar oleh kelas turunan melalui ctor atau cctor dilakukan melalui *constructor initialization list*

```
RetryingSender(std::string id, int maxRetries)
        : NotificationSender(std::move(id)),  // inisialisasi base
          maxRetries_(maxRetries) {}
```

Operator assignment

Assignment ditangani seperti inisialisasi.

- Jika kelas turunan **tidak** mendefinisikannya, operator= dari kelas dasar akan dipanggil (jika ada).
- Jika kelas turunan mendefinisikan operator= maka operasi *assignment* dari kelas dasar menjadi tanggung jawab kelas turunan.

Polymorphism

- Objek-objek dari kelas turunan memiliki sifat sebagai kelas tersebut dan sekaligus kelas dasarnya.
  - *polymorphism* (*poly* = banyak, *morph* = bentuk).
- Polymorphism memungkinkan program bekerja melalui tipe dasar, sementara perilaku aktual ditentukan oleh objek yang digunakan.
- Di C++, hal ini direalisasikan menggunakan reference atau pointer agar pemanggilan method dapat diputuskan saat run-time.
- Akibatnya ref / ptr memiliki tipe statik dan tipe dinamik.
  - Tipe statik = tipe objek pada saat deklarasikan,
  - Tipe dinamik = tipe objek pada saat eksekusi, dapat berubah bergantung pada objek yang diacu.

Dynamic binding

- Tipe dinamik digunakan pada saat eksekusi untuk memanggil method yang berasal dari kelas berbeda-beda melalui sebuah ref/ptr dari **kelas dasar.**
- Method yang akan dipanggil secara dinamik, harus dideklarasikan sebagai *virtual* (dilakukan di **kelas dasar**).
- Dalam contoh `NotificationSender` method `send()` akan dipanggil secara dinamik
  - Kelas `NotificationSender` harus mendeklarasikan sebagai method virtual

```
class NotificationSender {
  public:
    ...

    virtual void send(const std::string& recipient,
                      const std::string& message);

    ...
};
```

```cpp
#include "retrying_sender.hpp"

void funcVal(NotificationSender  s) { s.send("alice", "hello"); }

// memiliki tipe dinamik
void funcPtr(NotificationSender* t) { t->send("alice", "hello"); }

// memiliki tipe dinamik
void funcRef(NotificationSender& u) { u.send("alice", "hello"); }

void main() {
  RetryingSender rs;
  funcVal(rs);  // NotificationSender::send() dipanggil
  funcPtr(&rs); // RetryingSender::send() dipanggil
  funcRef(rs);  // RetryingSender::send() dipanggil
}
```

Virtual destructor

- Destructor di kelas dasar dideklarasikan virtual.
- Di dalam kelas turunan, destruktor **tidak perlu** dideklarasikan virtual karena otomatis mengikuti jenis dtor kelas dasar.

```cpp
class Base {
  public:
    virtual ~Base() { /* releases Base's resources */ }
};

class Derived: public Base {
  ~Derived() { /* releases Derived's resources */ }
};

int main() {
  Base* b = new Derived;
  delete b; // Makes a virtual function call to Base::~Base()
            // since it is virtual, it calls Derived::~Derived()
            // which can release resources of the derived class,
            // and then calls Base::~Base() following the usual
            // order of destruction
}
```

Abstract base class (ABC)

- *Abstract Base Class* = kelas dasar untuk objek abstrak.
  - Contoh: `DataStore`, `Vehicle`, `Shape`, dst.
- Bagaimana implementasi `Shape::Draw()`?
  - Tidak dapat diimplementasikan di dalam kelas `Shape`, namun harus dideklarasikan.
  - Dideklarasikan sebagai method *pure virtual*.
- ABC = kelas yang memiliki method *pure virtual*

Abstract base class

```
class List {
  public:
    // ...
    virtual void clear() = 0; // pure virtual
    // ...
};
```

- Tidak ada objek yang dapat dibuat dari kelas dasar abstrak
- Di dalam kelas non-abstrak (yang mewarisi kelas dasar abstrak) method *pure virtual* harus diimplementasikan.

```cpp
// File: LinkedList.hpp
// Deskripsi: deklarasi kelas LinkedList yang diturunkan dari
//            List

class LinkedList: public List {
  public:
    // ...
    void clear(); // punya impl. konkret untuk mengosongkan linked list
};

// File: ArrayList.hpp
// Deskripsi: deklarasi kelas ArrayList yang diturunkan dari List
class ArrayList: public List {
  public:
    // ...
    void clear(); // punya impl. konkret untuk mengosongkan array
};
```

Takeaway

- Inheritance dalam C++ digunakan untuk merealisasikan relasi spesialisasi yang telah dimodelkan secara konseptual.
- Melalui mekanisme bahasa seperti: virtual function, dynamic binding, konstruktor dan destruktor berantai, abstract base class,
- C++ memastikan bahwa:
  - objek turunan tetap dapat digunakan sebagai objek dasar,
  - perilaku yang tepat dijalankan sesuai jenis objek sebenarnya,
  - konsistensi makna dalam hierarki tetap terjaga pada saat program berjalan.
- Dengan demikian, inheritance bukan sekadar struktur kelas, melainkan cara menjaga agar variasi perilaku dalam model domain dapat bekerja secara benar di dalam program.